

FORMATION EMBARQUÉE

TP5 — Seance 4

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 4 : architecture FreeRTOS (3 h)

En-tête pédagogique

Objectifs	Structurer un firmware en tâches ; communiquer par queues ; ISR compatibles RTOS ; surveiller les piles ; valider par essais
Prérequis	Séances 1-3 ; module 06 §5 (FreeRTOS) ; module 10 §7
Outils	CubeIDE (FreeRTOS CMSIS-V2) ; le montage complet
Livrable	4 tâches communicantes + 4 essais de validation documentés

Déroulé minuté

0:00-0:25 — Activer FreeRTOS

1. CubeMX : `Middleware` → `FREERTOS` → interface CMSIS_V2.
2. **Point critique** : quand CubeMX le demande, déplacer la base de temps HAL de SysTick vers un timer dédié (ex. TIM10). Pourquoi ? SysTick appartient désormais à l'ordonnanceur RTOS ; la HAL a besoin de sa propre base pour `HAL_Delay` / timeouts. L'oublier = comportements erratiques.
3. Régler `configCHECK_FOR_STACK_OVERFLOW = 2` (détection pendant la mise au point).

0:25-0:45 — Déclarer les tâches et queues

Architecture imposée :

Tâche	Priorité	Période	Rôle
<code>T_Capteur</code>	normale	1 s	lit le BME280 → pousse une <code>mesure_t</code> dans <code>q_mesures</code>
<code>T_Traitement</code>	normale	sur message	moyennes, seuil → pousse vers <code>q_affichage</code>
<code>T_Console</code>	basse	sur caractère	console série (bloquée sur queue RX)
<code>T_Heartbeat</code>	la plus basse	500 ms	LED vie + surveillance des piles

Créer les tâches et deux `osMessageQueue` dans CubeMX (ou en code).

0:45-1:45 — Coder les tâches

Règle d'or : **toute communication par queue**, aucune variable globale partagée.

```

void T_Capteur(void *arg) {
    for (;;) {
        mesure_t m;
        if (bme280_lire(&capteur, &m.t, &m.p, &m.h))
            osMessageQueuePut(q_mesures, &m, 0, 0);
        osDelay(1000);           // rend le CPU pendant 1 s
    }
}

void T_Traitement(void *arg) {
    mesure_t m;
    for (;;) {
        if (osMessageQueueGet(q_mesures, &m, NULL, osWaitForever) == osOK) {
            // moyennes, comparaison seuil...
            osMessageQueuePut(q_affichage, &m, 0, 0);
        }
    }
}

```

Points clés : - `osDelay` (bloquant-coopératif) et non une attente active : les tâches de priorité inférieure tournent pendant ce temps. - **L'ISR UART** ne fait que pousser l'octet dans une queue avec la variante `...FromISR`. Et sa **priorité NVIC** doit être numériquement $\geq$ `configMAX_SYSCALL_INTERRUPT_PRIORITY` (module 10 §4.7) — sinon l'appel RTOS depuis l'ISR plante. C'est l'erreur de mise en route n°1. - Si tu partages l'I2C entre deux tâches, il faut un **mutex** ; ici une seule tâche (`T_Capteur`) y touche → pas besoin. **Explique ce choix** dans ton compte rendu (montrer qu'on sait quand un mutex est nécessaire vaut autant que d'en mettre un).

1:45-2:00 — Surveillance des piles

Dans `T_Heartbeat`, log périodiquement la marge de pile de chaque tâche :

```

printf("pile capteur=%lu traitement=%lu\r\n",
       osThreadGetStackSize(h_capteur), osThreadGetStackSize(h_traitement));

```

Et implémente le hook `vApplicationStackOverflowHook` : LED rouge fixe + `printf` du nom de la tâche fautive. Une pile trop petite est LE bug RTOS sournois (corruption aléatoire) — cette surveillance le rend visible.

2:00-2:50 — Les 4 essais de validation

#	Essai	Manip	Attendu
1	Endurance	laisser tourner 10 min	mesures stables, marges de pile stables (pas de dérive)
2	Capteur débranché en marche	retirer SDA	T_Capteur signale, les autres tâches vivent, reprise au rebranchement
3	Spam console	coller 1 Ko de texte	pas de crash, pas d'octets mélangés
4	Point d'arrêt dans T_Traitement 5 s puis reprise	debugger	le système rattrape : messages en attente traités

Essai 4 — le plus formateur : pendant la pause, `T_Capteur` continue de produire des mesures. Où vont-elles ? Dans `q_mesures`. Si la queue déborde (petite + pause longue), que se passe-t-il ?

`osMessageQueuePut` avec timeout 0 les **refuse** (retour $\neq$ `osOK`). **Documente ce comportement** : il n'y a pas une « bonne » réponse unique, mais il faut savoir laquelle tu as choisie (perdre les vieilles ? bloquer ? agrandir la queue ?) et pourquoi.

✅ **Point de contrôle** : les 4 essais documentés, essai 4 analysé.

2:50-3:00 — Barème + README de projet

Remplis la grille /20 du TP principal ([tp5-stm32-freertos.md](#)). Rédige le README du projet (schéma de câblage, photo, mode d'emploi console) — **c'est le projet à mettre en tête de ton portfolio firmware**. Commit final.

Erreurs fréquentes

Symptôme	Cause	Remède
Plante au démarrage RTOS	base de temps HAL restée sur SysTick	la mettre sur TIM10
<code>configASSERT</code> dans l'ISR UART	priorité NVIC trop haute (numériquement trop basse)	priorité $\geq$ <code>configMAX_SYSCALL...</code>
Corruption aléatoire de données	pile de tâche trop petite	augmenter, surveiller le high water mark
Une tâche affame les autres	attente active au lieu d' <code>osDelay</code>	<code>osDelay</code> / attente bloquante sur queue
Messages perdus en rafale	queue trop petite	dimensionner + choisir une politique

Bilan du TP 5 et de la formation

Ton dépôt contient maintenant : un driver I2C bare-metal écrit à la main, du DMA, une architecture RTOS propre avec queues et surveillance de pile, et des essais documentés. **C'est exactement ce qu'un**

recruteur firmware ouvre en premier. Avec les TP 1-4, tu couvres Arduino/IoT, FPGA, et les deux mondes automate — un profil « systèmes embarqués » complet.

Prochaine étape : un des [projets d'évaluation finale](#) (30-40 h) pour consolider, puis publie tout sur GitHub avec les barèmes remplis. Bon courage !

 Retour : [TP 5 \(vue d'ensemble\)](#) · [Parcours & ressources](#)